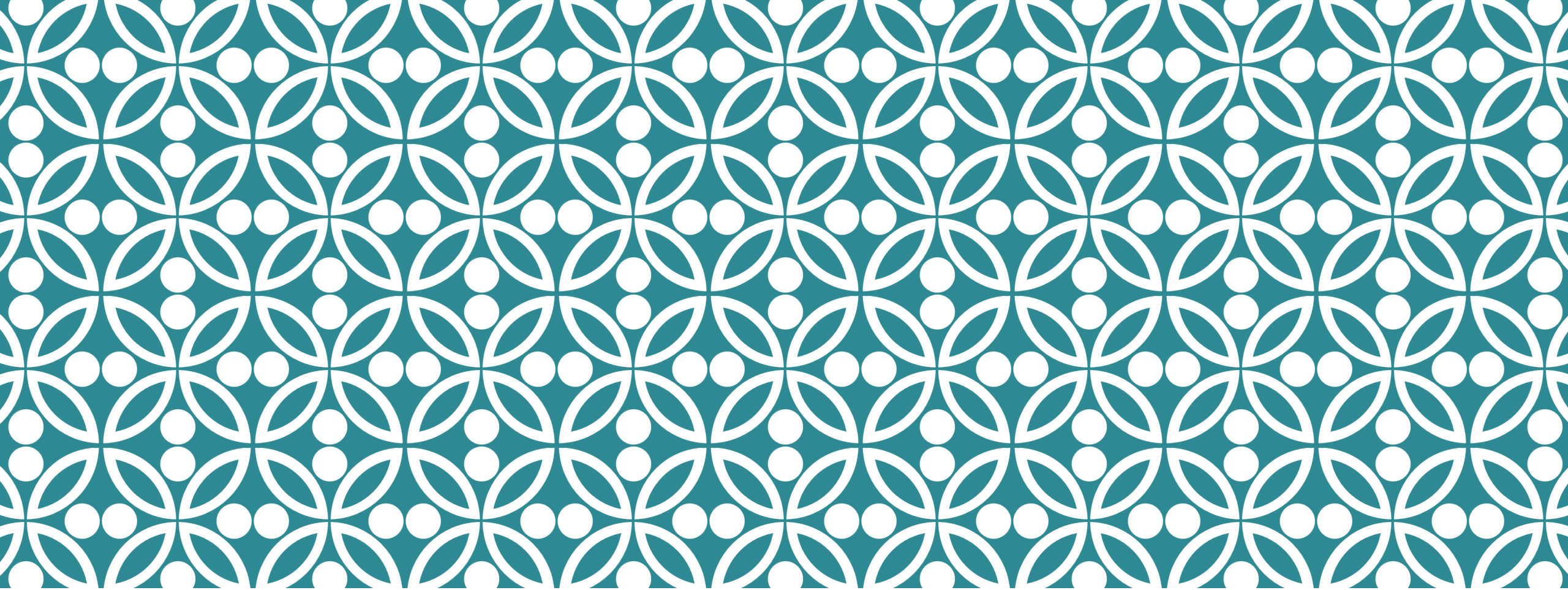


BASIC PROGRAMMING

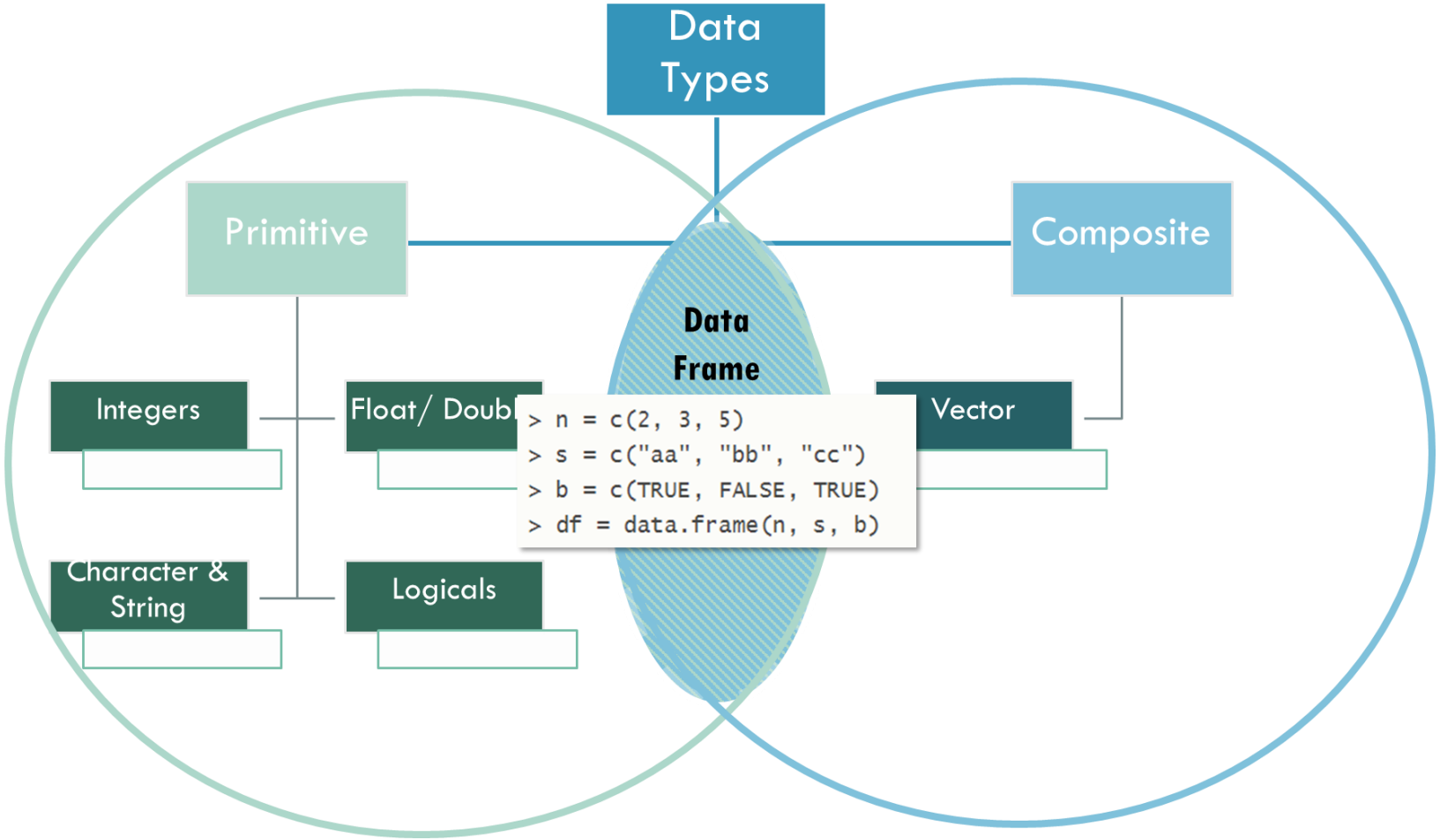
DR. SOFIA GIL-CLAVEL

1. Introduction
2. Using to APIs
3. Using function
4. Conditions and loops
5. Error handling

1. INTRODUCTION

- R and RStudio
- Data types
- Data frames

Introduction to R and Data frames



Introduction to R and Data frames

Using packages: Tidyverse

- Packages
- Tidy logic
- Dplyr

1.


```
> iris
# A tibble: 150 × 5
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
          <dbl>       <dbl>        <dbl>       <dbl> <fct>
 1          5.1         3.5          1.4         0.2 setosa
 2          4.9         3            1.4         0.2 setosa
 3          4.7         3.2          1.3         0.2 setosa
 4          4.6         3.1          1.5         0.2 setosa
 5          5           3.6          1.4         0.2 setosa
 6          5.4         3.9          1.7         0.4 setosa
 7          4.6         3.4          1.4         0.3 setosa
 8          5           3.4          1.5         0.2 setosa
 9          4.4         2.9          1.4         0.2 setosa
10          4.9         3.1          1.5         0.1 setosa
# … with more rows
# ℹ Use `print(n = ...)` to see more rows
```

2.

Basic piping

- `x %>% f` is equivalent to `f(x)`
- `x %>% f(y)` is equivalent to `f(x, y)`
- `x %>% f %>% g %>% h` is equivalent to `h(g(f(x)))`

3.

dplyr is a grammar of data manipulation, providing a consistent set of verbs that help you solve the most common data manipulation challenges:

- `mutate()` adds new variables that are functions of existing variables
- `select()` picks variables based on their names.
- `filter()` picks cases based on their values.
- `summarise()` reduces multiple values down to a single summary.
- `arrange()` changes the ordering of the rows.

Introduction to R and Data frames

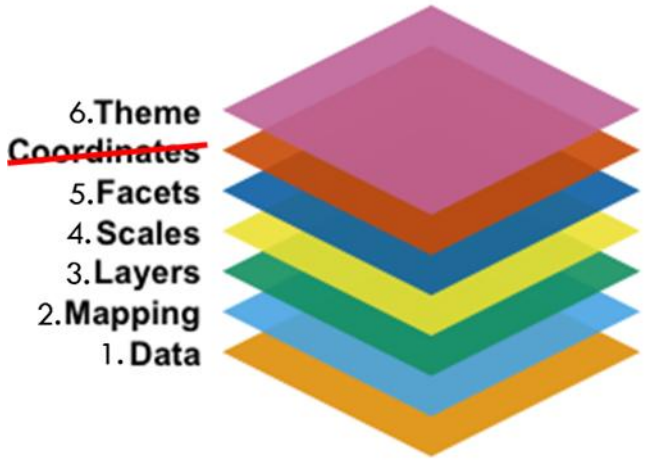
Using packages: Tidyverse

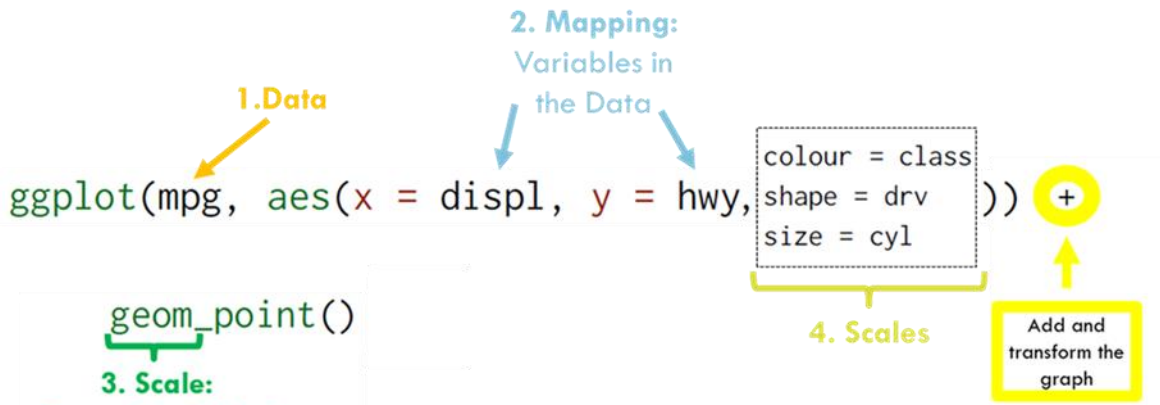
Data Handling

Data Handling

- Grammar of graphs
- Ggplot2
- Graphs components

Data Visualizations with ggplot2





Data Handling

Data Visualizations with ggplot2

Unsupervised Learning

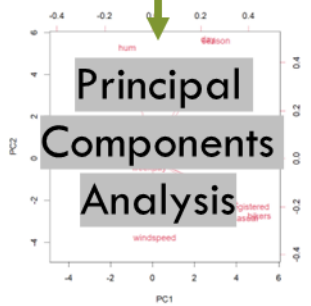
- 1.1 Statistical learning
- 1.2 Supervised vs. Unsupervised
- 1.3 PCA
- 1.4 K-means

$$\not{Y_i} \sim X1_i + X2_i + X3_i$$

Numeric Matrix

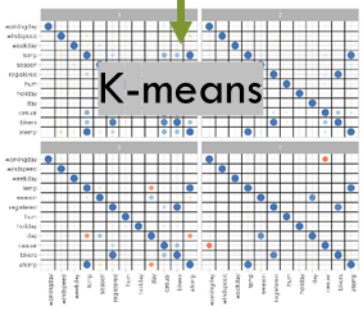
Dimensionality reduction

Principal Components Analysis



Clustering

K-means



Data Handling

Data Visualizations with ggplot2

Unsupervised Learning

Data Handling

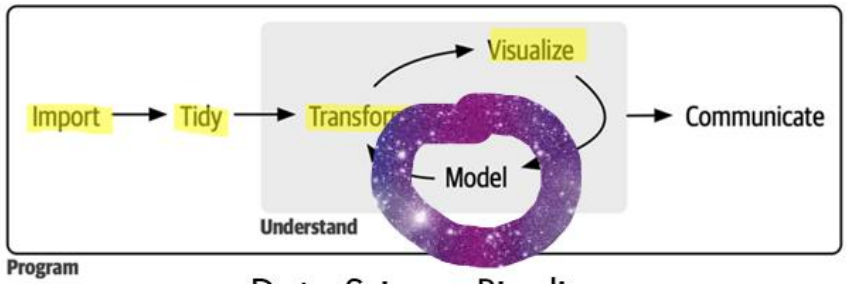
Exploratory Data Analysis

Data Handling

Exploratory Data Analysis

- Symbolic language
- Inference & prediction
- Fitting models
- Confidence intervals
- Statistical tests

Statistics



Data Science Pipeline

stats-package {stats} R Documentation

The R Stats Package

Description

R statistical functions

Details

This package contains functions for statistical calculations and random number generation.

For a complete list of functions, use `library(help = "stats")`.

Author(s)

R Core Team and contributors worldwide

Maintainer: R Core Team R-core@r-project.org

Data Handling

Exploratory Data Analysis

Statistics

Supervised Learning

- Supervised learning
- Assessing models and results

$$Y_i \sim X1_i + X2_i + X3_i \rightarrow$$

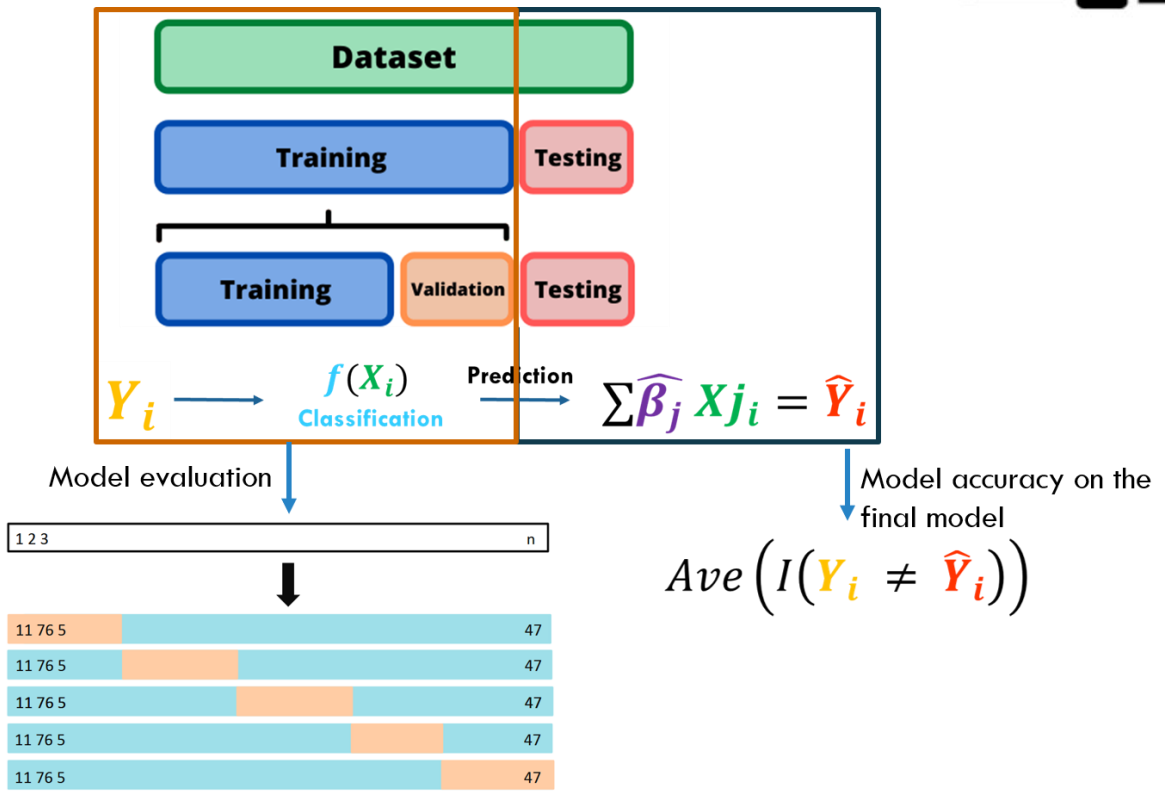


FIGURE 5.5. *A schematic display of 5-fold CV. A set of n observations is randomly split into five non-overlapping groups. Each of these fifths acts as a validation set (shown in beige), and the remainder as a training set (shown in blue). The test error is estimated by averaging the five resulting MSE estimates.*

Data Handling

Exploratory Data Analysis

Statistics

Supervised Learning

Data Handling

Exploratory Data Analysis

Machine Learning (ML)

Data Handling

Exploratory Data Analysis

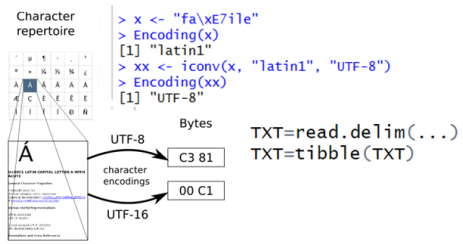
Machine Learning (ML)

- Basic function with Stringr
- Basic text handling
- Tidy text
- Basic DataViz

Text as Data

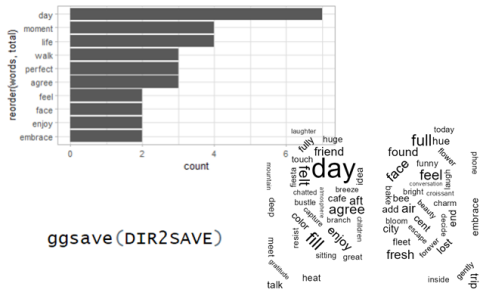
The pipeline

Right encoding

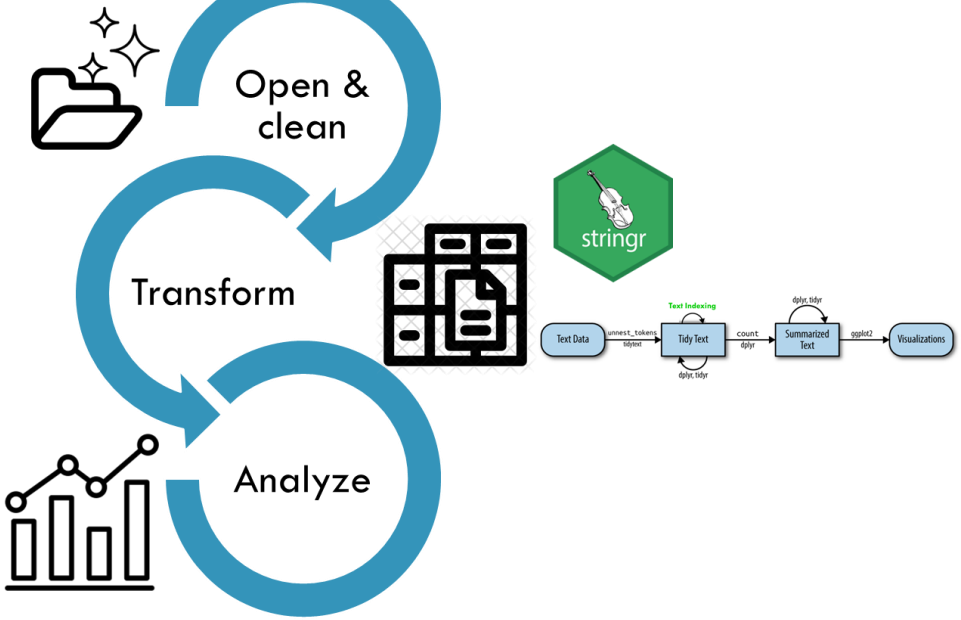



```
> x <- "fa\xE7ile"
> Encoding(x)
[1] "latin1"
> xx <- iconv(x, "latin1", "UTF-8")
> Encoding(xx)
[1] "UTF-8"
```

```
TXT=read.delim(...)
TXT=tibble(TXT)
```




```
ggsave(DIR2SAVE)
```

Data Handling

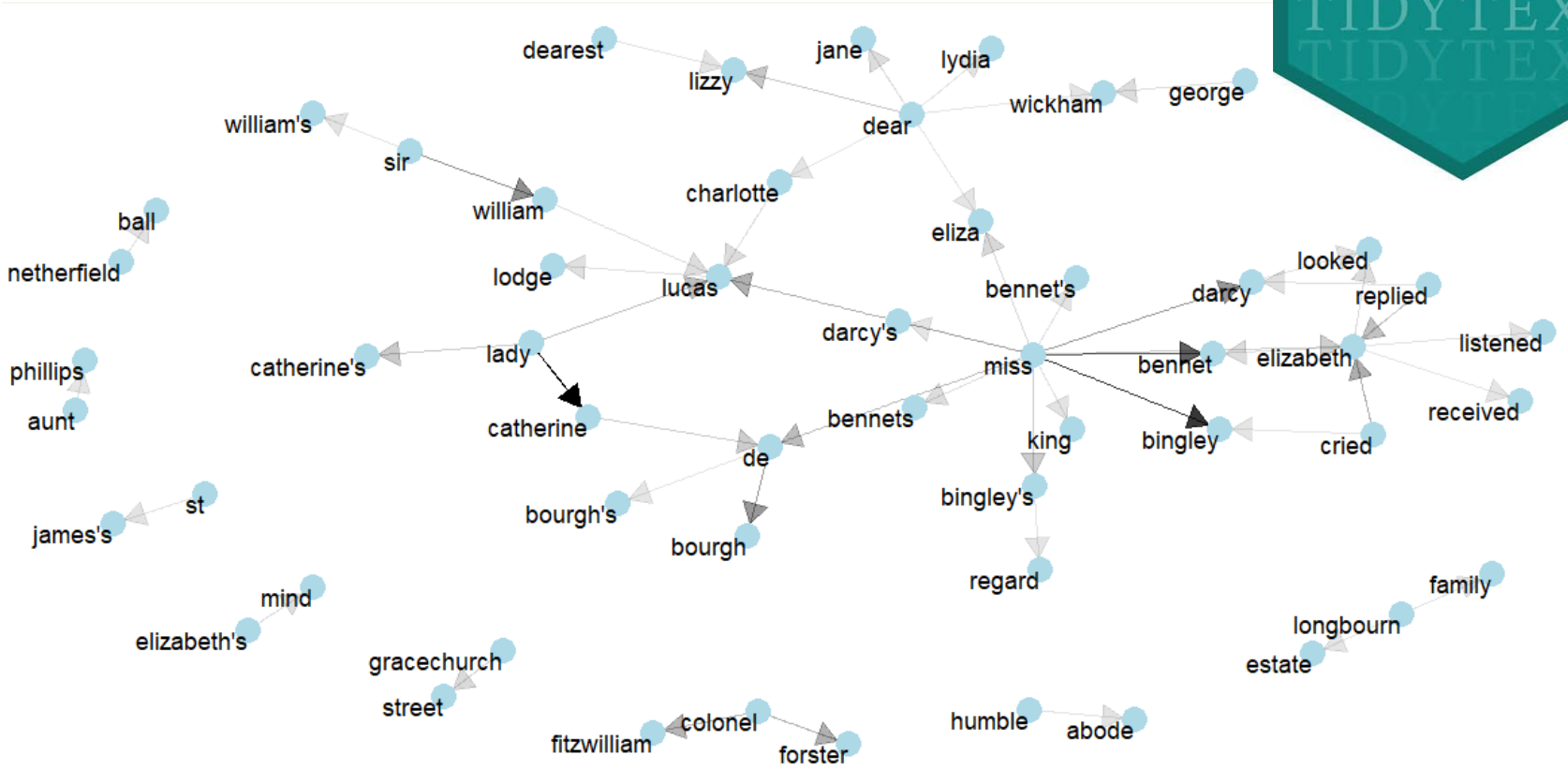
Exploratory Data Analysis

Machine Learning (ML)

Text as Data

Text Mining

- Word and document frequency
- Relationships between words
- Quick overview of NLP.



Data Handling

Exploratory Data Analysis

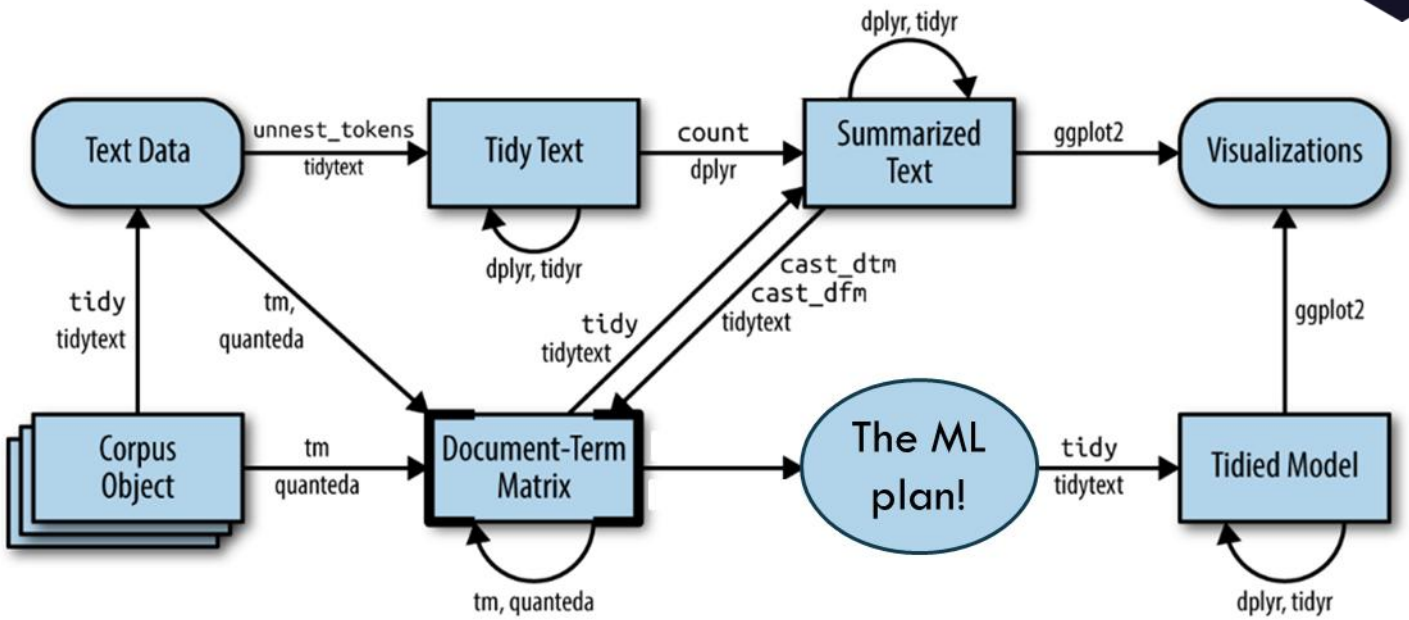
Machine Learning (ML)

Text as Data

Text Mining

- TextRecipes
- Unsupervised Methods for Text
- Tidymodels
- Supervised Methods for Text

ML for text



Data Handling

Exploratory Data Analysis

Machine Learning (ML)

Text as Data

Text Mining

ML for text

Data Handling

Exploratory Data Analysis

Machine Learning (ML)

Text Analysis

Data Handling

Exploratory Data Analysis

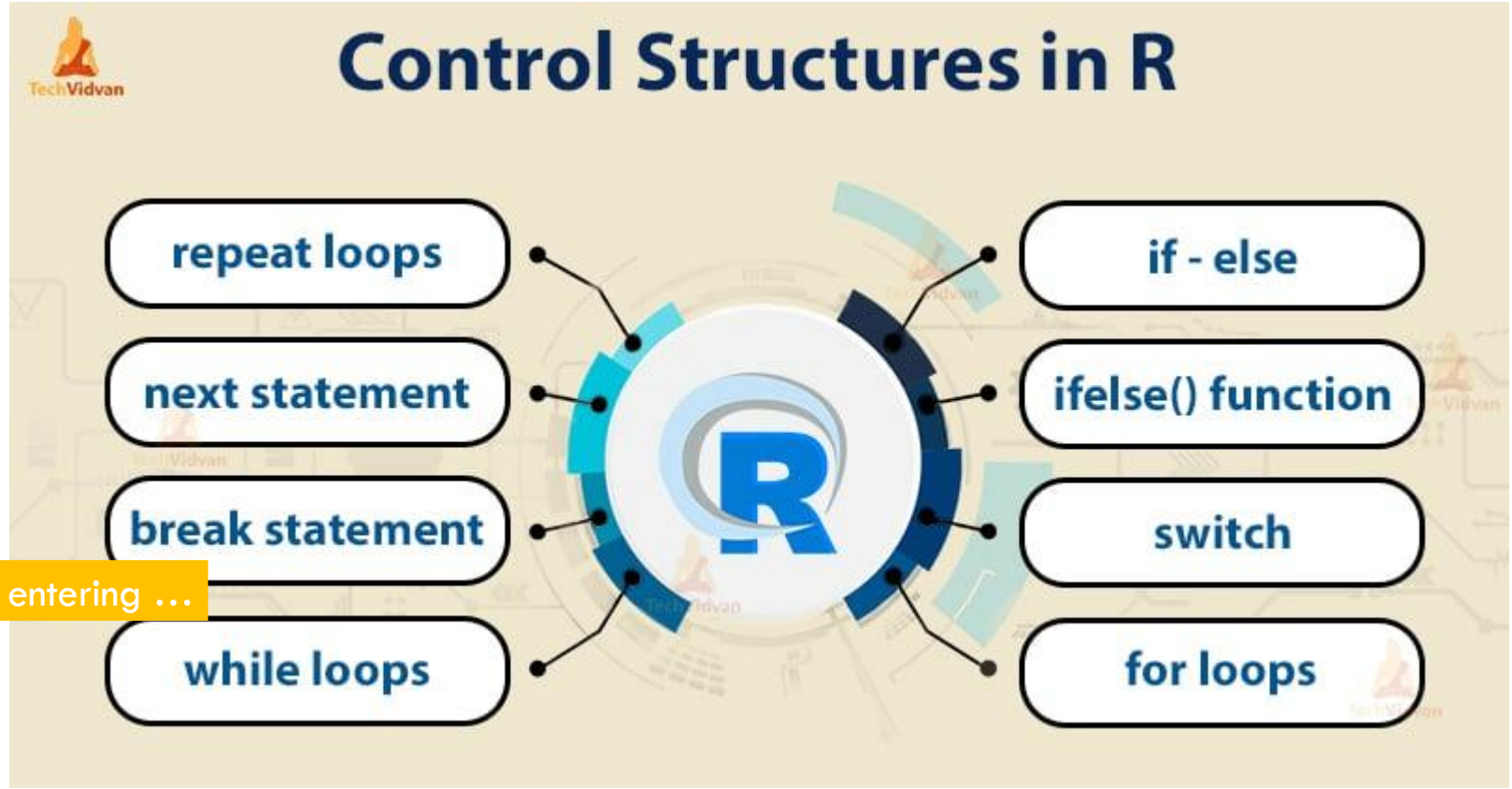
Machine Learning (ML)

Text Analysis

Basic Programming

- Using function
- Conditions and loops
- Error handling

We are entering …



Data Handling

Exploratory Data Analysis

Machine Learning (ML)

Text Analysis

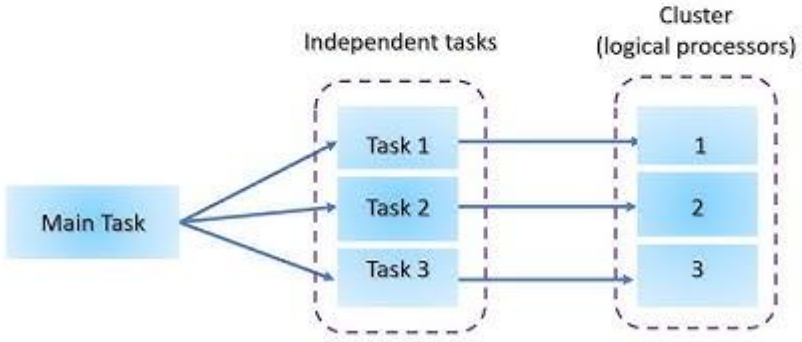
Basic Programming

- Intro to parallelization
- Parallelizing in R
- Servers: VU JupyterHub

Servers & Parallelization

Parallel Processing

Dr. Tam Nguyen



Data Handling

Exploratory Data Analysis

Machine Learning (ML)

Text Analysis

Basic Programming

Servers & Parallelization

Data Handling

Exploratory Data Analysis

Machine Learning (ML)

Text Analysis

Code Optimization

How does this all tie together?

With all this knowledge you can now analyze digital data!

Digital data from internet & social networks

Researchers retrieve the data via companies' APIs

Researchers keep the relevant data

Researchers analyse the data

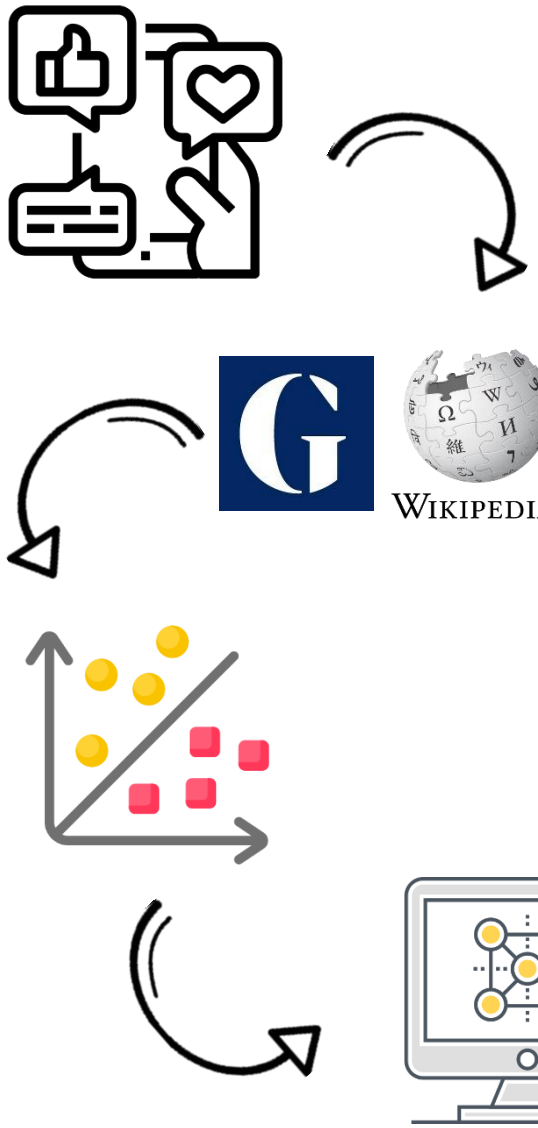
Icons from: https://freeicons.io/; and https://www.flaticon.com/

Digital data from internet & social networks

Researchers retrieve the data via companies' APIs

Researchers keep the relevant data

Researchers analyse the data



Code Optimization

Machine Learning (ML)

Text Analysis

Data Handling

Exploratory Data Analysis

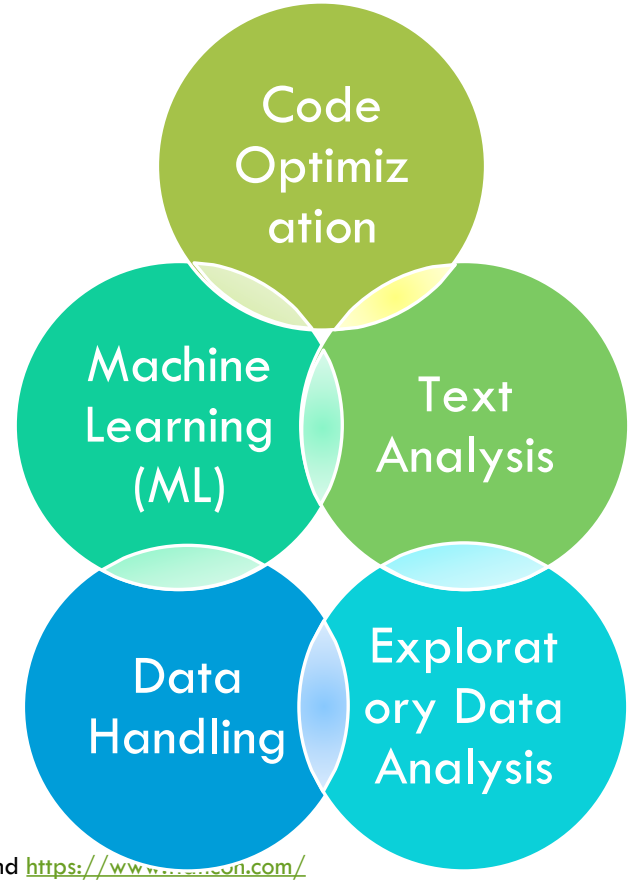
Icons from: https://freeicons.io/; and https://www.flaticon.com/

Digital data from internet & social networks

Researchers retrieve the data via companies' APIs

Researchers keep the relevant data

Researchers analyse the data



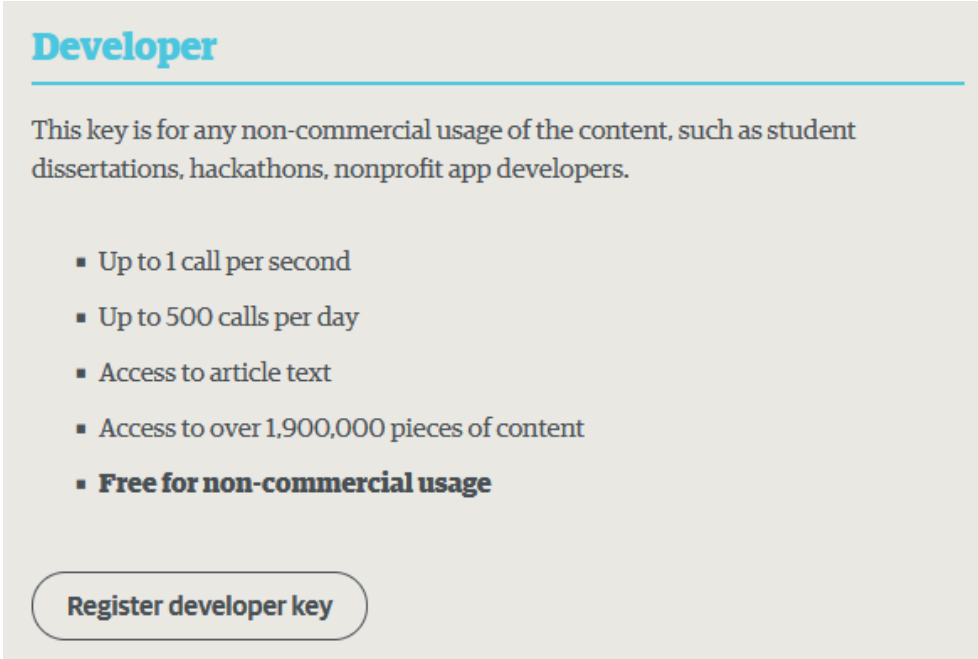
Icons from: https://freeicons.io/; and https://www.[illegible]con.com/

TODAY'S PLAN

Today, we are going learn to write function, conditions, and loops by applying them to retrieve the Guardian news about AI for the period between 2014 to 2026!

For this, you should have asked for a The Guardian developer token:

https://open-platform.theguardian.com/access/



1. INTRODUCTION TO APIS

Follows the same grammar as tidyverse!

httr2 (pronounced “hitter2”) is a comprehensive HTTP client that provides a modern, pipeable API for working with web APIs. It builds on top of {curl} to provide features like explicit request objects, built-in rate limiting & retry tooling, comprehensive OAuth support, and secure handling of secrets and credentials.

ACCESSING DATA VIA APIS

In general, an **Application Programming Interface (API)** is used for computer programs to communicate with each other. The most commonly use API type is the Representational State Transfer (REST). The REST API uses the HTTP protocol to send and receive standardized responses to a server.

There are five HTTP methods that you can use when making an API request to a server:

- GET: Retrieves data
- POST: Creates a new record.
- PUT: Modifies or replaces an entire record.
- PATCH: Modifies or updates parts of a record.
- DELETE: Deletes an entire record.

Source: https://raw.githack.com/Delft-RCafe/resources/main/themes/apis_in_r/apis-in-r.html

ACCESSING DATA VIA APIS

In general, an **Application Programming Interface (API)** is used for computer programs to communicate with each other. The most commonly use API type is the Representational State Transfer (REST). The REST API uses the HTTP protocol to send and receive standardized responses to a server.

There are five HTTP methods that you can use when making an API request to a server:

- **GET: Retrieves data**
- POST: Creates a new record.
- PUT: Modifies or replaces an entire record.
- PATCH: Modifies or updates parts of a record.
- DELETE: Deletes an entire record.

Source: https://raw.githack.com/Delft-RCafe/resources/main/themes/apis_in_r/apis-in-r.html

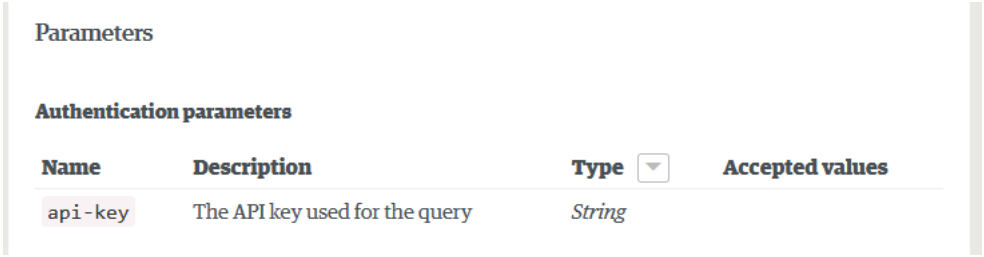
THE OTHER IMPORTANT COMPONENTS ARE:

- **endpoint**: The URL to find the resource you are trying to reach on the internet.
- **headers**: Provides information relevant both for us and for the server.
- **body**: These are specific parameters that the server requires.

For the Guardian API, we can look for all these components at:

https://open-platform.theguardian.com/documentation/search

- **endpoint**: https://content.guardianapis.com/search
- **headers:** Your the Guardian token/key.
- **body:** All the variables under the section "parameters"



Parameters

Authentication parameters

Name	Description	Type	Accepted values
`api-key`	The API key used for the query	*String*	

THE QUERY

To get data, we normally need to pass some search words that will constrain the data that we will retrieve.

You can check how to write this queries at: https://open-platform.theguardian.com/documentation/

Under “Query operators”:

> **Query operators**
>
> The `q` parameter supports AND, OR and NOT operators. For example:
>
> `debate AND economy` (https://content.guardianapis.com/search?q=debate%20AND%20economy&tag=politics/politics&from-date=2014-01-01&api-key=test) returns only content that contains both "debate" and "economy".

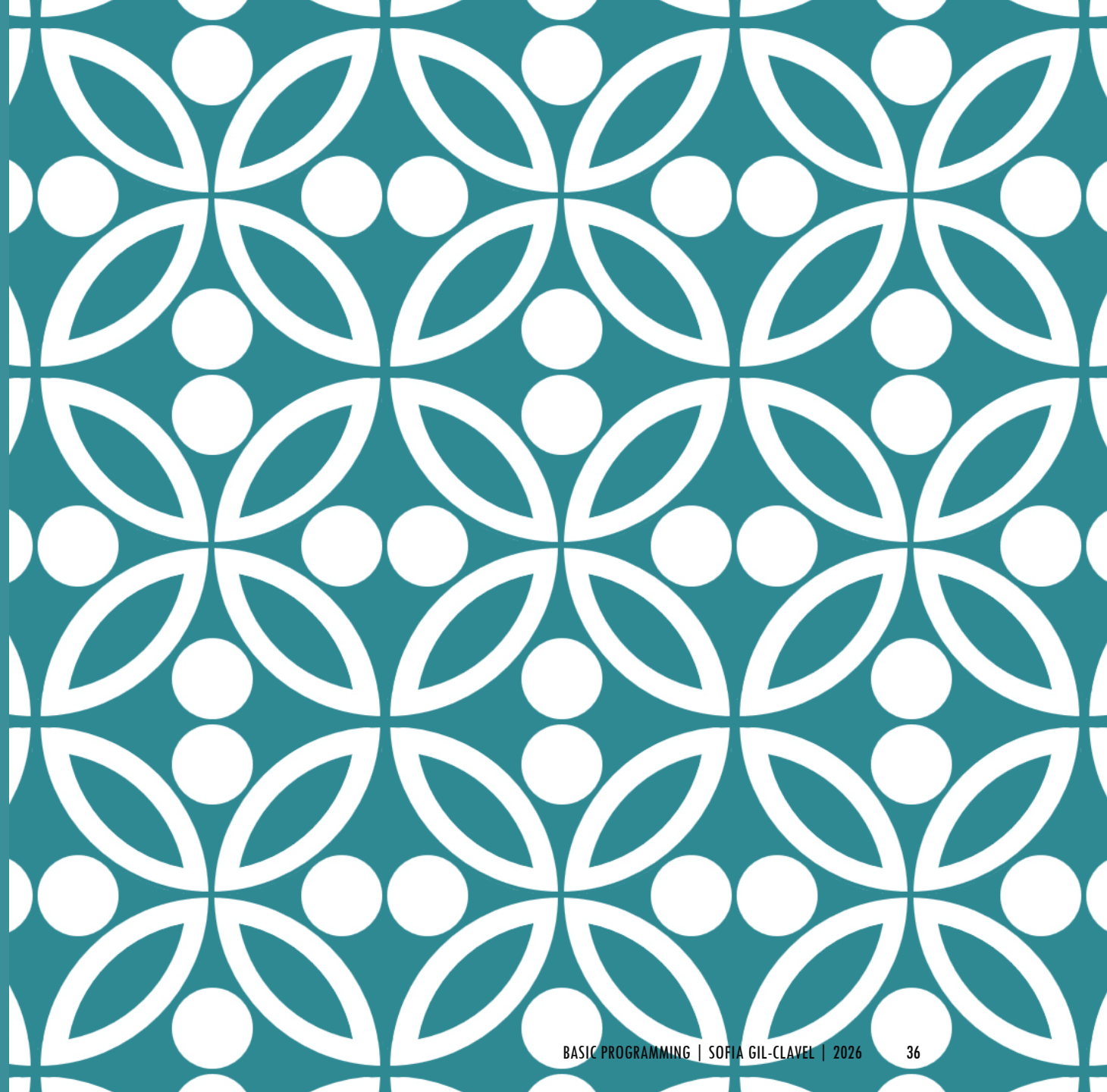
```
query="(artificial intelligence) OR (chat-gpt) OR (open AND ai) OR (llm) OR (large language model*)"
```

We have now assembled our first API retrieval.

But what if you want to retrieve more information?

Here is where functions, conditions, and loops are relevant!

2. WRITING FUNCTIONS



WRITING FUNCTIONS

When the same algorithm must be applied to different values, it is better to encapsulate the code into a function.

function {base} R Documentation

Function Definition

Description

These functions provide the base mechanisms for defining new functions in the **R** language.

Usage

```
function( arglist ) expr
\( arglist ) expr
return(value)
```

Arguments

`arglist`	empty or one or more (comma-separated) '`name`' or '`name = expression`' terms and/or the special token `...`.
`expr`	an expression.
`value`	an expression.

WRITING FUNCTIONS

When the same algorithm must be applied to different values, it is better to encapsulate the code into a function.

function {base} R Documentation

Function Definition

Description

These functions provide the base mechanisms for defining new functions in the **R** language.

Usage

```
function( arglist ) expr
\( arglist ) expr
return(value)
```

Arguments

`arglist`	empty or one or more (comma-separated) '`name`' or '`name = expression`' terms and/or the special token `...`.
`expr`	an expression.
`value`	an expression.

This is an ellipsis!
You can add parameters only used by functions inside your function.

EXERCISE 2.1:

Calculate the addition of all the numbers from 1 to n.

Function Specifications:

Input: n

Output: The addition

Hint: Use the functions seq() and sum().

EXERCISE 2.2

Turn the code to retrieve the Guardian data into a function with the following specifications.

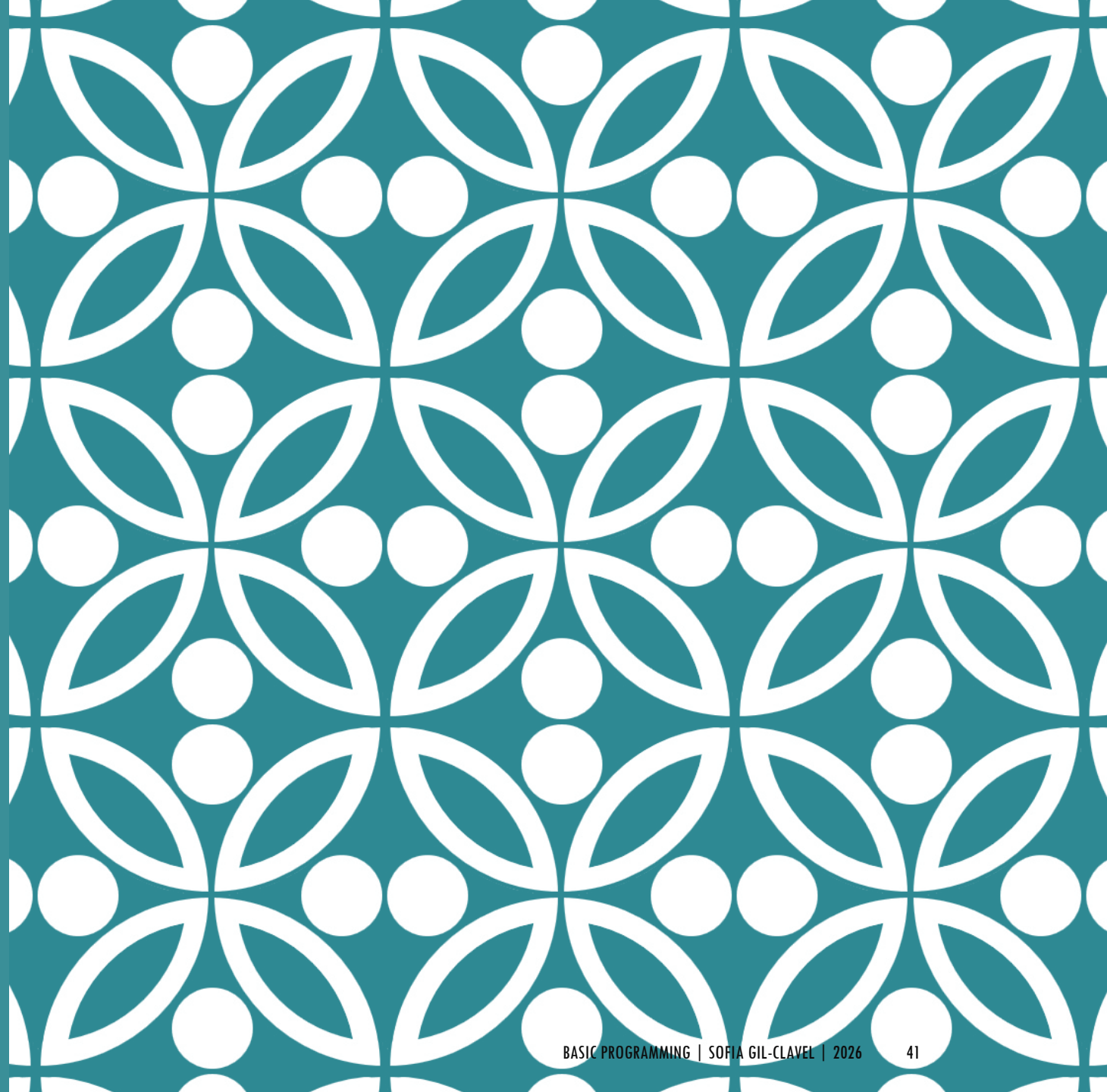
Parameters:

- query
- from_date
- to_date
- key

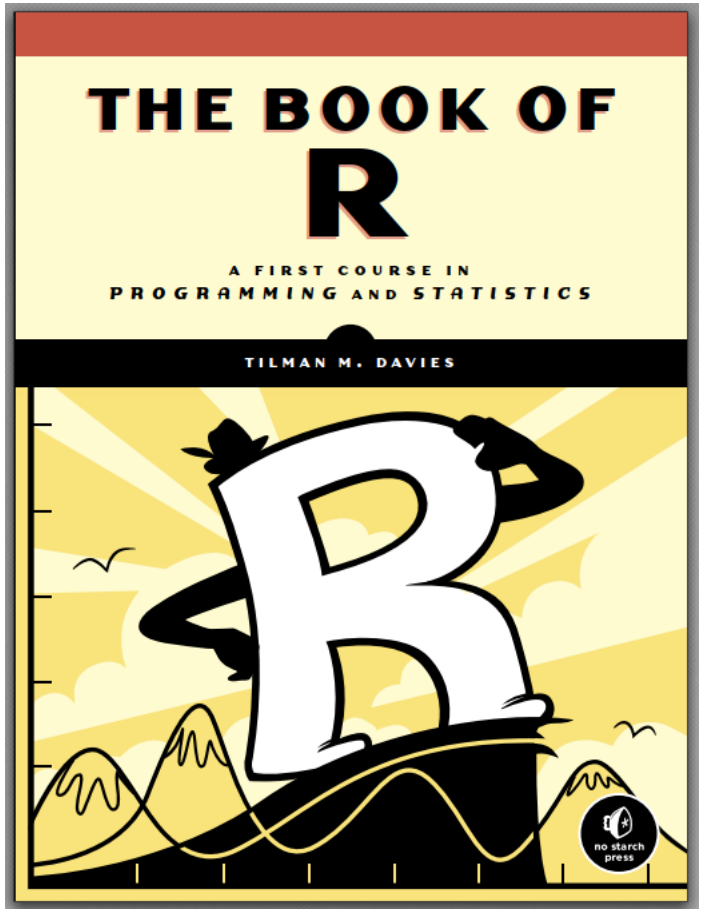
Output:

- The tibble of the Guardian response

3. CONDITIONS AND LOOPS



WE WILL FOLLOW:



Davies, Tilman M. ***The Book of R: A First Course in Programming and Statistics***. No Starch Press, 2016.

To write more sophisticated programs with R, you'll need to control the flow and order of execution in your code. One fundamental way to do this is to make the execution of certain sections of code dependent on a *condition*. Another basic control mechanism is the *loop*, which repeats a block of code a certain number of times. In this chapter, we'll explore these core programming techniques using `if-else` statements, `for` and `while` loops, and other control structures.

CONDITIONS

Inline if-else
If
If-Else
Nesting statements

INLINE IF-ELSE STATEMENTS

ifelse returns a value with the same shape as test which is filled with elements selected from either yes or no depending on whether the element of test is TRUE or FALSE.

ifelse(test, TRUE, FALSE)

test: an object which can be coerced to logical mode.

yes: return values for true elements of test.

no: return values for false elements of test.

➢ **Exercise 3.1:**

Create a vector that goes from 0 to 10.

Hint: Use the function seq().

Now, use the inline if-else statement to test which ones are bigger than 5.

If they are bigger than 5, then it should return "It is bigger than 5!"

Otherwise, it should return "No, it is not!"

IF STATEMENTS

The if statement is the key to controlling exactly which operations are carried out in a given chunk of code. An if statement runs a block of code only if a certain condition is true. These constructs allow a program to respond differently depending on whether a condition is TRUE or FALSE.

```
if(condition){
    do any code here
}
```

- **Exercise 3.2:**

  Write the code where you test if a variable is bigger than 3. If it is bigger than 3, then you print the string "It is bigger than 3!".

  Hint: Use the function print()

IF-ELSE STATEMENTS

The if statement executes a chunk of code if and only if a defined condition is TRUE. If you want something different to happen when the condition is FALSE, you can add an else declaration.

```
if(condition){
    do any code in here if condition is TRUE
} else {
    do any code in here if condition is FALSE
}
```

- **Exercise 3.3:**

  To the previous code, now add the condition that if the value is smaller or equal to 3 then you print "It is not bigger than 3!".

  Hint: Use the function print()

NESTING AND STACKING STATEMENTS

An if statement can itself be placed within the outcome of another if statement. By nesting or stacking several statements, you can weave intricate paths of decision-making by checking a number of conditions at various stages during execution.

```
if(mystring=="Homer"){
    foo <- 12
} else if(mystring=="Marge"){
    foo <- 34
} else if(mystring=="Bart"){
    foo <- 56
} else if(mystring=="Lisa"){
    foo <- 78
} else if(mystring=="Maggie"){
    foo <- 90
} else {
    foo <- NA
}
```

10 MINS BREAK

CODING LOOPS

for loop
while loop

FOR: REPEATS CODE AS IT WORKS ITS WAY THROUGH A VECTOR, ELEMENT BY ELEMENT.

The R for loop always takes the following general form:

```
for(loopindex in loopvector){
    do any code in here
}
```

```
for(i in 1:5) {
  print(i)
}
```

Exercise 3.4:

Create a for loop to print the letters from “a” to “e”.

WHILE: SIMPLY REPEATS CODE UNTIL A SPECIFIC CONDITION EVALUATES TO FALSE.

A while loop uses a single logical-valued loopcondition to control how many times it repeats.

Upon execution, the loopcondition is evaluated. If the condition is found to be TRUE, the braced area code is executed line by line as usual until complete, at which point the loopcondition is checked again.

The loop terminates only when the condition evaluates to FALSE, and it does so immediately—the braced code is not run one last time.

```
while(loopcondition){
    do any code in here
}
```

Exercise 3.5: Add the value 1 ten times.

1. In the variable count save the integer 0.
2. Create a while loop in which you add the value 1 to the variable count.
3. The while loop should run while the variable count is smaller than 10.

REPEAT

Another option for repeating a set of operations is the repeat statement.

Notice that a repeat statement doesn't include any kind of loopindex or loopcondition. To stop repeating the code inside the braces, **you must use a break declaration inside the braced area (usually within an if statement);** without it, the braced code will repeat without end, creating an infinite loop. To avoid this, you must make sure the operations will at some point cause the loop to reach a break.

```
repeat{
    do any code in here
break
}
```

To end the loop it is necessary to use the special variable "break".

Exercise 3.6:

Write the code in the previous exercise replacing the "while" with "repeat" and "break".

Hint: Place the break inside an if statement.

IN SUMMARY

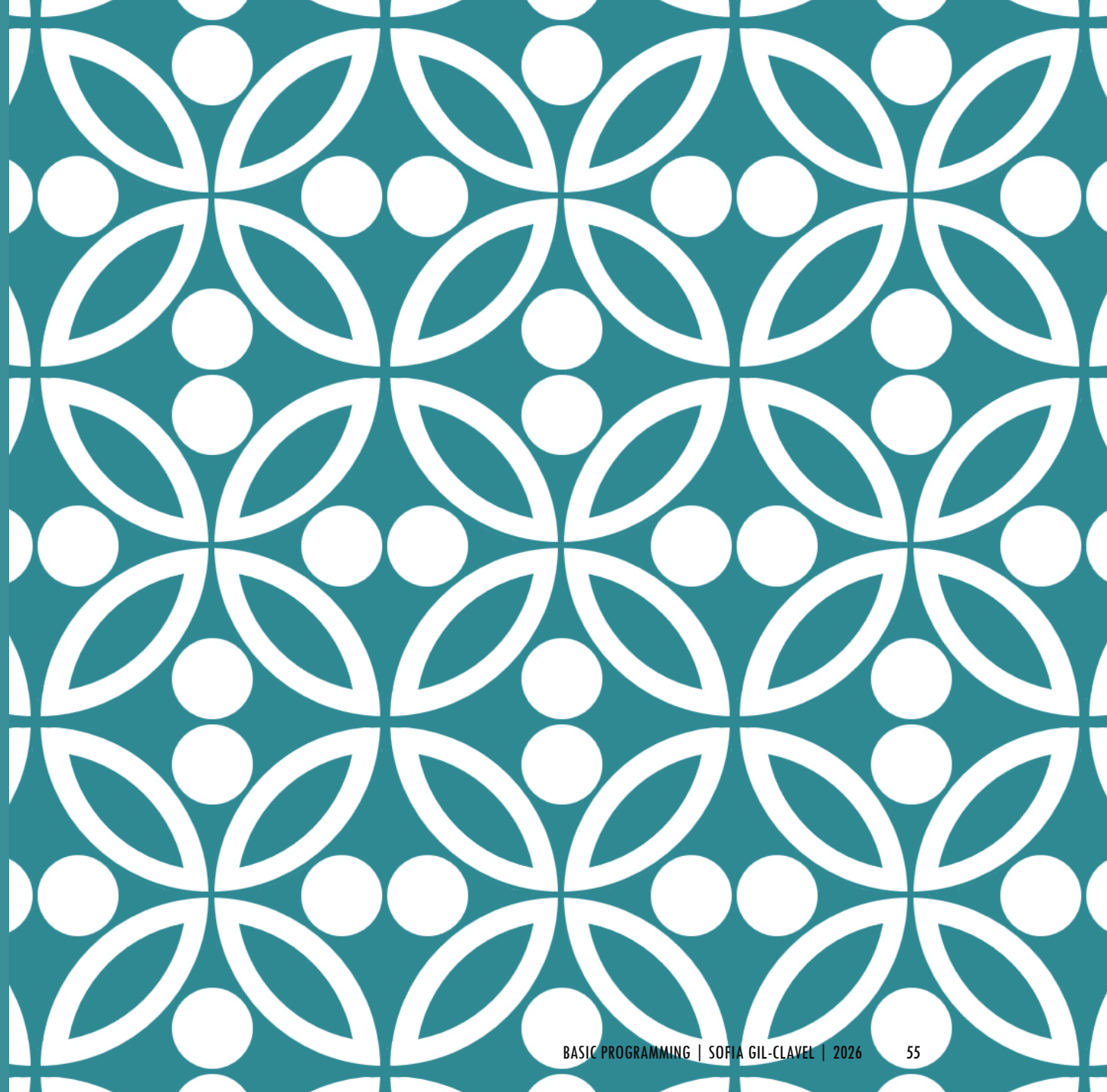
Function/operator	Brief description
`if( ){ }`	Conditional check
`if( ){ } else { }`	Check and alternative
`ifelse`	Element-wise `if-else` check
`break`	Exit explicit loop
`next`	Skip to next loop iteration
`repeat{ }`	Repeat code until `break`

The control sequences are divided into:

- Bifurcation statements:
  - If/else - The if statement is the key to controlling exactly which operations are carried out in a given chunk of code.
  - Loops:
    - for: Runs a loop a finite number of times.
    - while: Runs a loop as long as a condition is true
  - Other control mechanisms:
    - repeat (break): The repeat structure executes a loop infinitely. The only way to end the loop is to call the break function inside it.

Control structures describe the order in which statements or groups of statements are executed.

4. ERROR HANDLING



ERRORS CAN HAPPEN

When an API fails, it will return a 4xx value error. You can find more possible errors here: https://guardian.hedera.com/guardian/readme/api-guideline

Some of these errors include:

1. You use an invalid key. This one is error 401.
2. You exceed the rate limit. This one is error 429.

YOU USE AN INVALID KEY

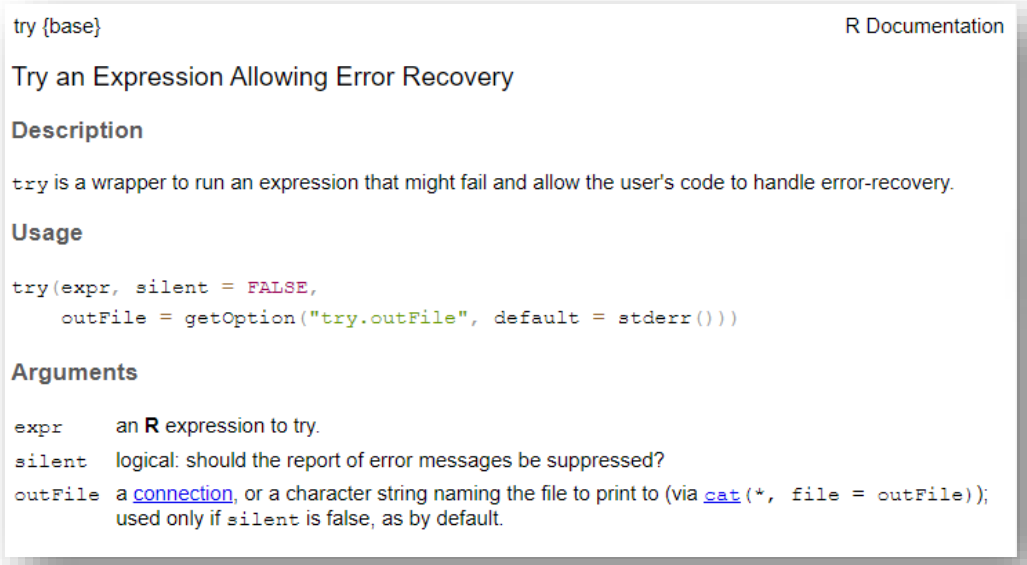
Delete the last two digits of your the Guardian key and save it in the variable KEY2.

Run your RETRIEVAL function with the wrong key. You should get an error like:

```
> RETRIEVAL(query,"2014-01-01", "2014-06-01", key = KEY2)
Error in `req_perform()`:
! HTTP 401 Unauthorized.
Run `rlang::last_trace()` to see where the error occurred.
```

THE TRY() FUNCTION

To handle the errors, we need to first detect them. For this, we will use the function try().



try {base} R Documentation

Try an Expression Allowing Error Recovery

Description

`try` is a wrapper to run an expression that might fail and allow the user's code to handle error-recovery.

Usage

```
try(expr, silent = FALSE,
    outFile = getOption("try.outFile", default = stderr()))
```

Arguments

`expr`	an **R** expression to try.
`silent`	logical: should the report of error messages be suppressed?
`outFile`	a connection, or a character string naming the file to print to (via `cat(*, file = outFile)`); used only if `silent` is false, as by default.

Exercise 4.1:

Write an if statement that reacts to the function RETRIEVAL if it returns the 401 error.

Hint 1: Pass the following line as a parameter of try() and save the result in the variable message.

```
resp<-RETRIEVAL(query,"2014-01-01", "2014-06-01", key = KEY2)
```

If so, it prints the message generated by the try().

Hint 2: Use the function str_detect().

YOU EXCEED YOUR RATE LIMIT

API rate limits 175 views

Jon Wade Apr 15, 2016, 3:29:43 PM

to Guardian Open Platform API Forum

Hey there - I've built an analysis app (http://jonwade.digital/hosted-projects/guardian-api) on the back of your API as part of a project I am doing for a coding academy. It's working great, but I upped the times the API is being polled (trying to get back more articles) and now it seems I'm rate limited and getting a status 429 back from the server. What is the API poll limit per day? Is it possible to increase it for this project? If I know what the limit is, I can write some logic into the app to ensure it doesn't exceed that amount per period. Let me know (and hope you like the app!!). JW

Clinton Yeboah May 12, 2016, 9:29:26 PM

to Guardian Open Platform API Forum

Hi John, if you are looking for the Guardian API rate limit, this is it:

Developer: Not more than 12 calls per second, not more than 5000 calls per day -- FREE

Commercial: According to your plan!

according to this link: http://open-platform.theguardian.com/access/

And according to another link here: you can ask for some more limit if you contact TheGuardian and you are able to convince them: http://open-platform.theguardian.com/documentation/

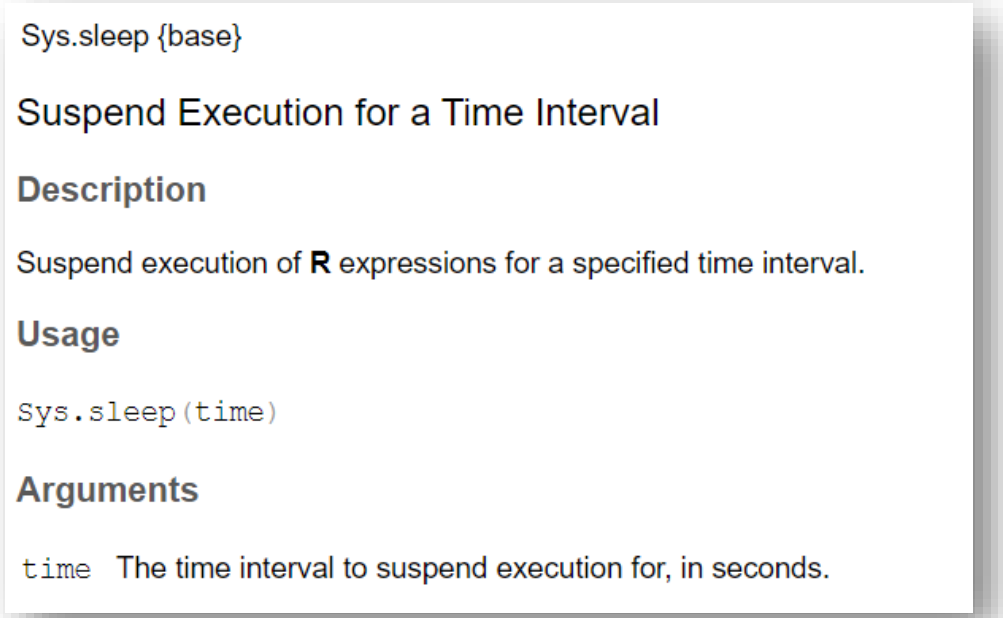
Good luck bro.

So, you just have to wait!

https://groups.google.com/g/guardian-api-talk/c/X-v_2f_xwfI/m/uXqwOQPkLAAJ

THE SYS.SLEEP() FUNCTION

Some errors are managed by waiting for the API to reset. For those errors, you can use Sys.sleep().



Sys.sleep {base}

Suspend Execution for a Time Interval

Description

Suspend execution of **R** expressions for a specified time interval.

Usage

```
Sys.sleep(time)
```

Arguments

`time`	The time interval to suspend execution for, in seconds.

Exercise:

1. Turn the previous if statement into a nested if-else that also reacts to the function RETRIEVAL if it returns the 429 error.

   Hint: Use the functions try() and str_detect().

2. If so, it waits for 5 mins (300 seconds).

EXERCISE 4.3:

Let's use while to continue the loop once the waiting has finished:

1. Place the nested conditions inside a while loop.
2. The while loop breaks when the class() is not a "try-error" type anymore.

RETRIEVING IN PRODUCTION

Now that we know how to build functions, write conditions and loops, and how to handle errors, it is time to put everything in production!

Let's retrieve The Guardian news in 6-months intervals:

- "from-date" = "2014-01-01", "to-date" = "2014-06-30"
- "from-date" = "2014-07-01", "to-date" = "2014-12-31"
- "from-date" = "2015-01-01", "to-date" = "2015-06-30"
- "from-date" = "2015-07-01", "to-date" = "2015-12-31"
- …
- "from-date" = "2026-01-01", "to-date" = " 2026-06-30 "

How would you do that?

- **Hint:** Use the function paste0() to create the vectors "from-date" and "to-date". Once you have the vectors, use a for loop to pass the values into the RETRIEVAL function. Remember to leave the error handling outside the function, those ones go inside the loop.

[hybrid]R-Workshop: Servers and Code-parallelization in R

This is the last workshop in the R-series. The final session will focus on using servers for parallel processing. During this session, we will use all the learnings from “Basic concepts before code-parallelization in R” to speed up codes using computing servers.

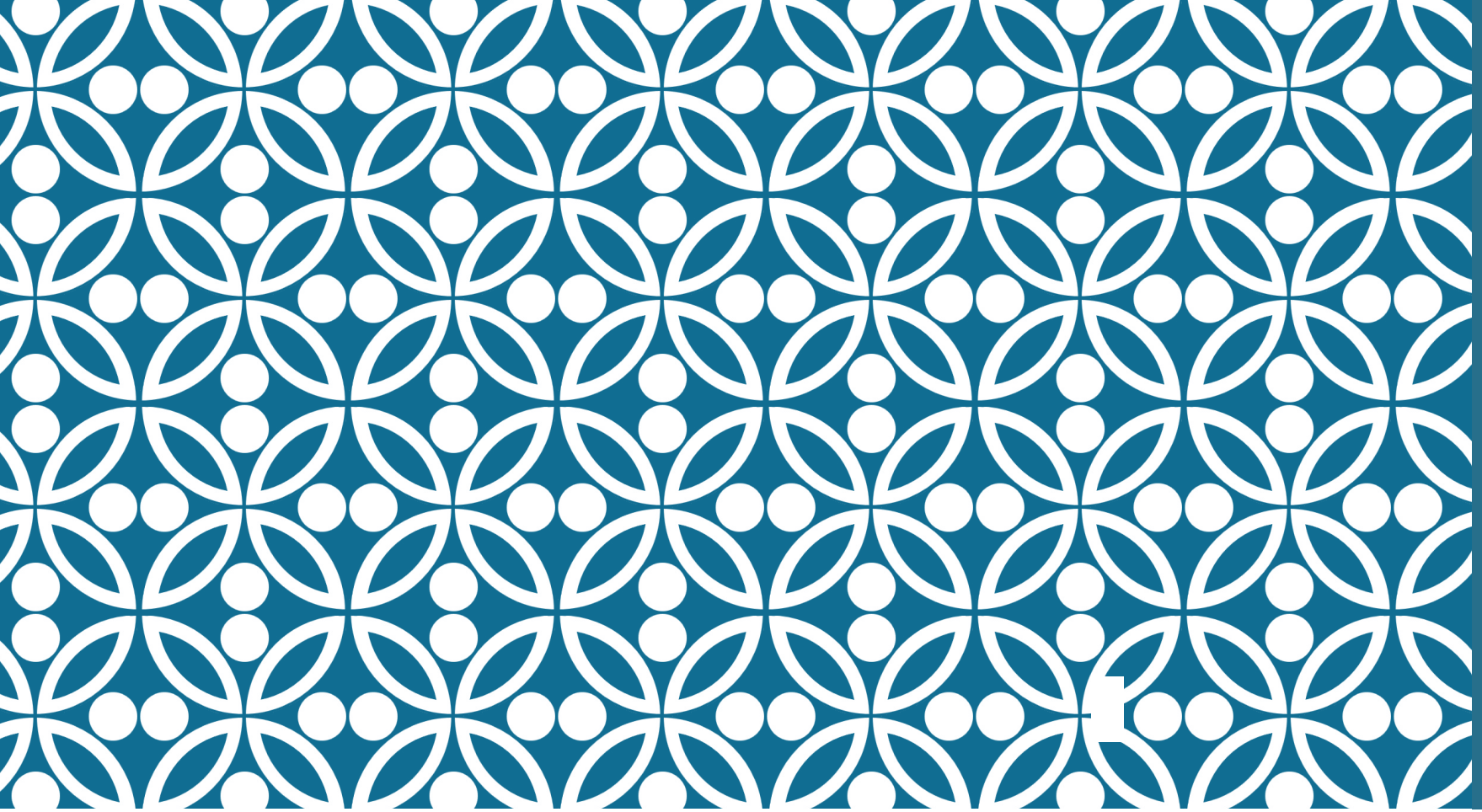
https://forms.office.com/e/8Bgd2YsasJ

All about the lab:

https://societal-analytics.nl/

Contact us at:

analytics-lab.fsw@vu.nl.



https://sofiag1l.github.io/

THANKS!

Dr. Sofia Gil-Clavel